

NNsight and NDIF: Democratizing Access to Open-Weight Foundation Model Internals

Jaden Fiotto-Kaufman*, Alexander R. Loftus*, Eric Todd, Jannik Brinkmann^{1,2}, Koyena Pal, Dmitrii Troitskii, Michael Ripa, Adam Belfki, Can Rager³, Caden Juang, Aaron Mueller, Samuel Marks, Arnab Sen Sharma, Francesca Lucchetti, Nikhil Prakash, Carla Brodley, Arjun Guha, Jonathan Bell, Byron C. Wallace, David Bau

¹Northeastern University, ²TU Clausthal, ³University of Hamburg

ABSTRACT

We introduce NNsight and NDIF, technologies that work in tandem to enable scientific study of very large neural networks. NNsight is an open-source system that extends PyTorch to introduce deferred remote execution. The National Deep Inference Fabric (NDIF) is a scalable inference service that executes NNsight requests, allowing users to share GPU resources and pretrained models. These technologies are enabled by the *intervention graph*, an architecture developed to decouple experimental design from model runtime. Together, this framework provides transparent and efficient access to the internals of deep neural networks such as very large language models (LLMs) without imposing the cost or complexity of hosting customized models individually. We conduct a quantitative survey of the machine learning literature that reveals a growing gap in the study of the internals of large-scale AI. We demonstrate the design and use of our framework to address this gap by enabling a range of research methods on huge models. Finally, we conduct benchmarks to compare performance with previous approaches. Code, documentation, and tutorials are available at <https://nnsight.net/>.

1 INTRODUCTION

Research on large-scale AI currently faces two practical challenges: *lack of transparent model access*, and *lack of adequate computational resources*. Model access is limited by the secrecy of state-of-the-art commercial model parameters (OpenAI et al., 2023; Anthropic, 2024; Gemini Team et al., 2024), and computational resources are limited by funding and engineering barriers. Commercial application programming interfaces (APIs) help amortize costs by offering frontier models “as a service”. However, these APIs lack the transparency necessary to enable scientists to study model internals, e.g., by providing access to intermediate activations or gradients used during neural network inference and training.

Surveys of AI research needs have documented this situation (Shevlane, 2022; Casper et al., 2024). Bucknall & Trager (2023) highlight the need for *structured model access APIs* that offer greater transparency than existing commercial inference APIs. The current paper aims to meet this need by defining an expressive and efficient technical standard that addresses the key problems for providing a practical and useful structured model access API to support research on large models.

Our work makes three contributions:

The *intervention graph* architecture (Figure 1b), an approach for organizing experiments on very large models that reduces engineering burden, enhances reproducibility, and enables low-cost communication with remote models. We compare the intervention graph architecture to classical approaches for running similar experiments, and show how it achieves these goals.

NNsight (Figure 1a), an open-source implementation of the intervention graph architecture that extends PyTorch (Paszke et al., 2019) to create an expressive programming idiom that supports transparent model interventions on large-scale AI without requiring users to store or manage model parameters locally. NNsight builds intervention graphs using the same deferred computation graph

*Equal contribution. Correspondence to: {j.fiotto-kaufman, loftus.al}@northeastern.edu

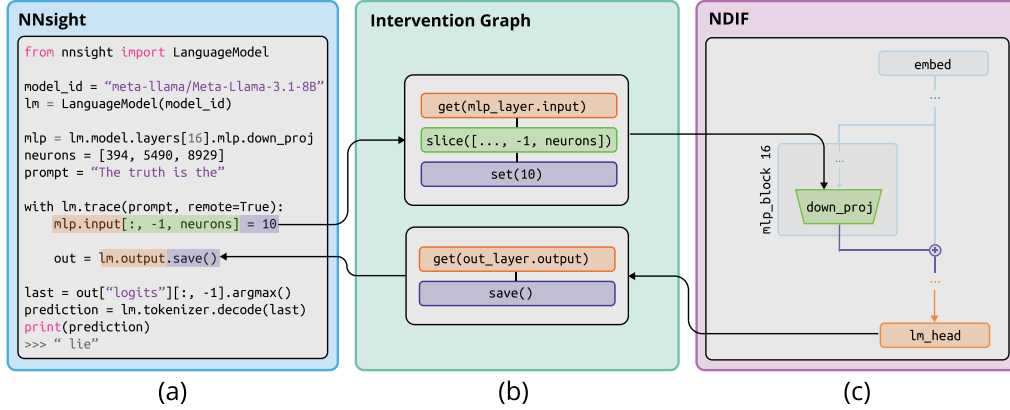


Figure 1: An example of the implementation of an NNsight intervention graph: (a) A user writes research code from which (b) an intervention graph is constructed. (c) The intervention operations are interleaved with the original model’s computation and then executed. Values marked with `.save()` are made available to the user upon completion.

idiom that deep learning frameworks adopt to enable automatic differentiation (Bottou & Le Cun, 1988; Bottou & Gallinari, 1990; Abadi et al., 2016; Al-Rfou et al., 2016; Bradbury et al., 2018).

NDIF (Figure 1c), an open-source cloud inference service that supports this idiom by providing behind-the-scenes user sharing of model instances to reduce the costs of large-scale AI research. Crucially, by executing intervention graphs from multiple users with safe co-tenancy, NDIF supports the expressiveness of NNsight while avoiding costly startup times and communication costs incurred by traditional high-performance computing (HPC). We measure the performance of NDIF against the traditional approach, and we also compare it with Petals (Borzunov et al., 2023), a peer-to-peer approach to reducing barriers to distributed computing with large AI models.

2 SURVEYING MODEL AVAILABILITY AND RESEARCH USAGE

Compared to commercial APIs, models with openly downloadable parameters enable much more invasive experimentation and are a valuable resource for interpretability researchers (Scao et al., 2022; Biderman et al., 2023; Touvron et al., 2023; Jiang et al., 2023). To understand current research activity studying the internals of these open-weight models, we examine a set of 184 papers sourced from a recent survey of interpretability research (Ferrando et al., 2024) — see Appendix A for data curation details.

In Figure 2, we visualize the research usage of the largest open-weight models studied in these papers over time. While there has been a rapid investment in *training* larger and more capable models, we find that there is a disparity between the most capable systems¹ and those being *studied* in detail. There is a small group of papers that study language models with $\geq 70\%$ MMLU performance (see (a) in Figure 2), suggesting that this gap can partially be explained as a previous lack of sufficiently capable models. However, even after the release of more-capable open-weight models, 60.6% of the surveyed research papers since February 2023 are still doing research on smaller, less performant models ($< 40\%$ MMLU). This suggests that the gap is not just due to a lack of capable models, but may also be due to engineering and infrastructural barriers.

Studying models with up to 70 billion parameters is possible for labs with access to high-end compute such as NVIDIA A100 or H100 80GB GPUs, but doing research *beyond* this scale represents a leap in technical complexity and resource requirements. For example, inference on a 530 billion parameter LLM uses 1TB of GPU memory, requiring many devices distributed across multiple nodes just to load model parameters in 16-bit precision (Aminabadi et al., 2022; Dettmers et al., 2023). Even if a scientist has access to such resources, they must integrate their experiment code with model sharding, parallelization, quantization, and other deployment specifics. Thus, despite evidence that

¹Defined in terms of MMLU performance (Hendrycks et al., 2021, see Appendix A for details.)

Figure 2: Most interpretability research is done on models that lag far behind the capabilities available in either closed- or open-models. Each blue point represents the MMLU performance of the largest open-weight model studied by a surveyed paper, where the size of the point represents the model’s parameter size. Models without a recorded MMLU score were interpolated with nearest neighbors. There is a significant gap between the performance of models studied (blue line) and the capabilities of leading open-weight models (shown in orange). This gap is extended even further when considering the performance of leading closed-weight models (black line). (a) A small group of papers study language models with $\geq 70\%$ MMLU performance, which can account for at least some of the gap. However, many researchers are still studying smaller, less performant models that hover around baseline performance (shown in gray). (b) Studying smaller, but still capable models such as Qwen 72B or Yi-34B may be part of the solution to closing this research gap, but these models still underperform the leading open-weight model, Llama 3.1 405B.

many fascinating capabilities appear only in the largest models (Brown et al., 2020; Rae et al., 2021; Wei et al., 2022; Patel & Pavlick, 2022; Schaeffer et al., 2023), studying them can seem unattainable.

One solution is to study similarly capable, smaller models; this approach is used in two recent works that study Yi-34B (Young et al., 2024) and Qwen 2 72B (Yang et al., 2024) (see (b) in Figure 2) — these are models that use distillation techniques to compress larger model capabilities into smaller model sizes. However, these models still underperform the most capable open-weight models, such as Llama 3.1 405B (Dubey et al., 2024). In the following sections we describe how NNsight and NDIF are designed to address this research gap by providing a flexible system for studying very large open models, presenting a reusable and modular service suitable for fully-transparent large-scale deployment distributed across many devices and nodes.

3 A FRAMEWORK FOR EXPERIMENTS ON LARGE-SCALE AI

Research on the internals of pretrained neural networks follows one common pattern: Experimental code is inserted between computational steps of the network, interleaved with code previously written to run the network. Depending on the experiment, researchers’ code may inspect internal representations (Bau et al., 2017), perform interventions (Vig et al., 2020), collect gradients (Sundararajan et al., 2017; Selvaraju et al., 2017; Ancona et al., 2019; Bastings et al., 2022; Kramár et al., 2024), modify parameters (Meng et al., 2022), or use and train the parameters of supplementary models (Lester et al., 2021; Wu et al., 2023; Huben et al., 2024; Templeton et al., 2024; Marks et al., 2024).

Traditionally, researchers have organized their code by creating bespoke versions of neural network modifications and hooks for each experiment using callback systems (Li et al., 2024; Wang et al., 2022; Geva et al., 2023, and see Figure 3a), or by designing custom neural network systems with

built-in access points (Nanda & Bloom, 2022; Wu et al., 2024b). While tying experiments with network implementations is primarily a matter of code organization for small models, it burdens researchers with the responsibility and cost of model deployment and storage, increasing engineering challenges as parameter sizes increase. This pattern also makes it difficult for external research groups to reproduce large distributed neural network experiments without duplicating the environment.

We introduce a new framework that implements the following pattern: Separate experimental and engineering code, store the experiment, and then execute it on shared computational infrastructure. We achieve this separation by expressing an experiment as an *intervention graph* that can be saved, transmitted, optimized, and interleaved with model execution.

This framework implements three core requirements necessary for scalable, reproducible research that are not currently met by classical approaches:

- **Separation:** The framework must decouple experimental and engineering code, so that the increasingly complex challenge of running the underlying model can be tackled separately from experimental design.
- **Expressiveness:** The framework must allow the user to express all the experiments that they would have been able to express if they were using PyTorch on local hardware.
- **Co-Tenancy:** Experiments must be able to be safely, efficiently, and concurrently combined with model execution at runtime, to allow the service to amortize costs across many users.

Each of these requirements is addressed by one of the aspects of our framework: First, the *intervention graph* allows us to decouple research and engineering code. Next, the *NNsight API* is designed to be fully expressive, because users define interventions using Python and PyTorch code. Finally, the *NDIF inference server* enables co-tenancy, by setting up a central system to which experiments can be safely transmitted, optimized, and run concurrently.

3.1 REPRESENTING EXPERIMENTS AS GRAPHS

Our core innovation is a portable, serializable representation of an experiment, called the *intervention graph*, which dynamically captures modifications to the model’s computation graph. This graph can be stored in JSON format, version-controlled, optimized, and sent to or retrieved from remote systems for execution. In this section, we formalize the notion of an intervention graph.

Computation Graph. Our formalism adopts Al-Rfou et al. (2016, Theano)’s definition of a computation graph². The computation graph is a weakly connected, bipartite, directed, acyclic graph defined as $C = (V, A, E)$, where V and A are *variable nodes* and *apply nodes*, respectively. E is a set of directed edges connecting the variable nodes and the apply nodes. The definition and properties of V , A , and E are as follows:

- $V = \{v_1, \dots, v_m\}$ is the set of variable nodes representing objects. They are one-to-many, and are connected to apply nodes.
- $A = \{a_1, \dots, a_n\}$ is the set of apply nodes representing operations on variable nodes. They are many-to-one, and are connected to variable nodes.
- $E = \{e_1, \dots, e_p\}$ is the set of directed edges. An edge e in E either connects a variable node v to an apply node a , or vice versa. Thus, $E \subseteq (V \times A) \cup (A \times V)$ — making C a bipartite graph as there are no connections between nodes of the same type.

In our context, a node that is “one-to-many” has a single input edge, but multiple output edges, as seen with variable nodes. Likewise, a node that is “many-to-one” has multiple input edges but only a single output edge, as with apply nodes. In NNsight, apply nodes correspond to PyTorch Modules.

The graph C is *weakly connected* since converting all directed edges to undirected edges results in a connected graph. It is *bipartite* because edges only exist between variable and apply nodes.

²Al-Rfou et al. (2016) allow apply nodes to have many outgoing edges, whereas in ours, apply nodes can only have one outgoing edge. Appendix E argues that the two definitions are equivalent for our use case.

Figure 3: Experiment code expressed using (a) standard PyTorch hooks and (b) the NNsight API. Both code snippets define the same intervention – activating three neurons which cause the model to invert the meaning of its output (e.g., producing “lie” rather than “truth”). The PyTorch intervention code captured in six lines of code (a, lines 8-13), can be easily expressed using NNsight with three lines of code (b, lines 7-9). Standard PyTorch requires creating custom hooks for each access point, whereas with NNsight, all module inputs and outputs can be accessed within a single trace context.

Intervention Graph. *Interventions* are modifications to the computation graph C that introduce additional nodes and edges. We define an intervention component $C' = (V', A', E')$, where C' is a computation graph representing a single user-specified edit. We define an intervention as $I = \{C', \mathcal{G}, \mathcal{S}\}$, where sets $\mathcal{G}$ and $\mathcal{S}$, referred to as *getters* and *setters*, determine how C' connects to C . Integrating an intervention component into C is called *interleaving*, and consists of two parts:

- The *getters* ($\mathcal{G}$) connect variable nodes in C to apply nodes in C' , i.e., $\mathcal{G} \subseteq V \times A'$.
- The *setters* ($\mathcal{S}$) connect variable nodes in C' back to apply nodes in C , i.e., $\mathcal{S} \subseteq V' \times A$.

The *intervention graph* of NNsight is the union of all individual intervention components, $\mathcal{I} = \bigcup_{1 \leq i \leq k} C'_i$. The final augmented computation graph is constructed by interleaving each intervention component with the original graph, and represents a complete user experiment. Appendix B.1 discusses implementation details of how intervention graphs are constructed, and Figure 10 (Appendix E) shows an example of an abstract intervention graph.

For the intervention graph to be valid, for every *getter* edge $(v_i, a'_j) \in \mathcal{G}$ and *setter* edge $(v'_k, a_l) \in \mathcal{S}$, there must be no directed path from a_l to v_i or from v'_k to a'_j . This rule ensures that the augmented computation graph is acyclic.

3.2 FAMILIAR AND EXPRESSIVE INTERVENTIONS

To define and execute intervention graphs in code, the NNsight API uses PyTorch notation inside of a Python context window. We enable the tracing of user-defined experiments inside the context window by overriding basic Python operations as well as wrapping every PyTorch module and submodule to expose their inputs, outputs, and gradients. We call this the *tracing context*.

In Figure 3, we compare two code snippets that define the same intervention — one using standard PyTorch hooks (Figure 3a) and the other using NNsight’s API (Figure 3b). In both snippets, a language model (Llama-3.1-8B; Dubey et al., 2024) is loaded from HuggingFace (Wolf et al., 2020) and an intervention is specified – activating three neurons which cause the model to invert its output (e.g., producing “lie” instead of “truth”). While using standard PyTorch hooks requires creating custom hooks for each access point (Figure 3a, lines 8-13), NNsight’s trace context window provides access to the intermediate inputs and outputs of all PyTorch modules (Figure 3b, lines 7-9). See Appendix C for additional NNsight code examples.

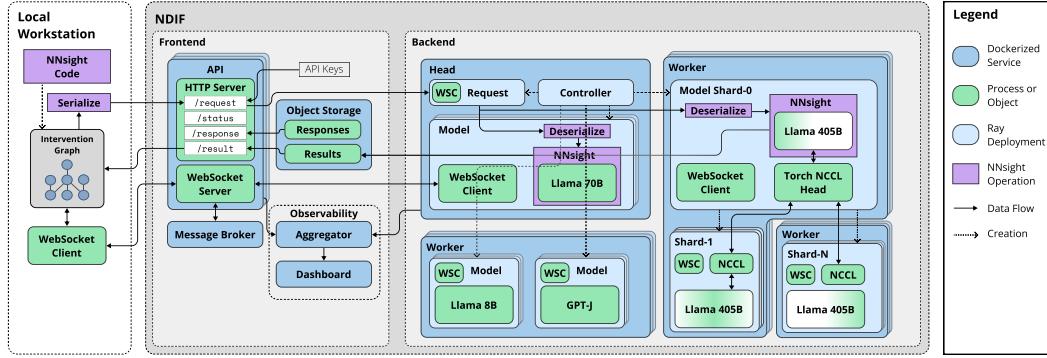


Figure 4: An overview of the NNsight and NDIF remote system. Researchers write experiment code using the NNsight API which is converted to an Intervention Graph. The graph is serialized to a custom JSON format and sent as a request to the NDIF frontend server. The NDIF backend can host multiple model instances, each on a dedicated set of GPU nodes. For very large models, such as Llama 3.1 405B (Dubey et al., 2024), model weights are distributed across many shards using tensor parallelism. The router transfers the request to the head node (shard 0) of the requested model, via the Ray GCS Service (Moritz et al., 2018; Ray Team, 2022). Shard 0 sends the request to all other shards of the model where it is then deserialized and executed. Each shard receives the full intervention graph, but only manages a slice of the model parameters. The Torch NCCL Head manages distributed model execution across allocated shards. After the intervention graph has been executed, results are gathered at shard 0 and sent to the object store in the NDIF frontend. The shard 0 WebSocket client informs the WebSocket client on the local workstation about the completion of the intervention. As soon as the local WebSocket client notes the intervention is complete it pulls the final results from the Object Store and inserts the result back into the local intervention graph. The research code can pull results from the intervention graph that are requested via `.save()`.

The experiment’s corresponding intervention graph is defined upon exiting the context window, and execution does not occur until the context is complete. The graph can then either be executed locally, or it can be serialized and sent to a remote system for execution. Tensors within the deferred execution block are not directly available to the main code, but every deferred tensor provides a `.save()` method to make its data available to the main program. Although the tracing context defers execution, it is still possible to debug many issues locally by using the PyTorch FakeTensor system, which precomputes and checks tensor shapes and datatypes while building the computation graph.

Because PyTorch code is used inside the tracing context, any experiment that can be written using standard PyTorch hooks can also be expressed using NNsight and will feel familiar to PyTorch users. NNsight wraps all 217 fundamental PyTorch tensor operations, supports built-in modules and custom neural network architectures, and includes infrastructure for loading models distributed by popular platforms such as HuggingFace. Users can express a wide range of interventions beyond simple probing and manipulation, including custom attention mechanisms, dynamic computation graph alterations, weight updates, and training loops (see Code Example 5), as well as computing gradients on backwards passes and gradient flow modifications.

By separating experiment code from the underlying neural network implementation, NNsight allows remote execution of experiments by simply adding a `remote=True` flag (Figure 3b, line 7), which sends the experiment to NDIF for execution.

3.3 CO-TENANCY AND REMOTE INFRASTRUCTURE

The NDIF service, in contrast to the NNsight system, is a multi-user runtime infrastructure designed to amortize costs and remove engineering burden from scientists by concurrently sharing inference. NDIF accepts intervention graphs created by NNsight, and executes them on large, preloaded models. In Figure 4, we provide an overview of the different components of the system, their responsibilities, and how they interact with one another.

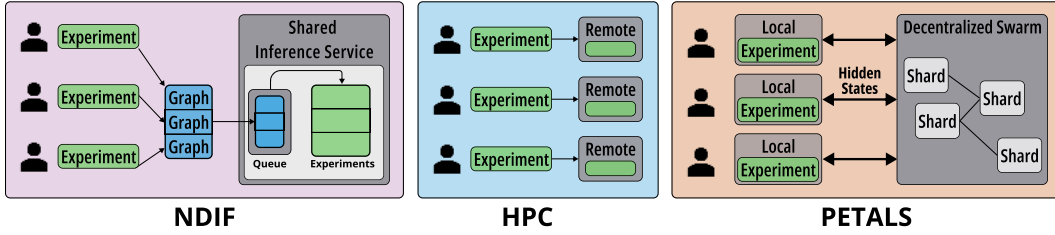


Figure 5: Schematic of research community use of NDIF vs. HPC and Petals. Green nodes show custom experiments. NDIF (left) allows many researchers to share a common inference service that runs customized experiments with shared memory. In HPC (center), researchers are responsible for weight-loading and handling model memory overhead on their own separate instances. In peer-to-peer swarm approaches like Petals (right), while GPU resources are shared, hidden states must be transferred between nodes during inference and returned to the user for custom interventions, resulting in costly data transfers.

Unlike traditional distributed computing approaches for supporting research experiments in AI, the NDIF infrastructure is designed to minimize the communication overhead involved in remote model execution. Figure 5 compares the infrastructure to traditional approaches.

In high-performance computing services (HPC), shared computational resources are allocated to researchers at the level of machines or virtual machines. That means that when researchers conduct experiments on customized AI models, all of the engineering code, weight-loading, and model storage must be handled by the researcher on instances that are exclusively allocated to them.

One approach that has been proposed for sharing costly computational resources while retaining researcher control is to distribute work using a peer-to-peer distributed service. This is the approach supported by Petals (Borzunov et al., 2023), which provides a swarm of inference servers that preload and provide inference computation services for layers of large neural networks. The Petals architecture can support custom experiment work by enabling researchers to host their own customized intervention code for particular layers on their own nodes (e.g., on their own workstation) while relying on the swarm for other inference steps. However, this architecture incurs costly data transfers multiple times during a forward pass for experiments that intervene on the inference process.

NDIF is designed to execute intervention graphs within the high-performance cluster itself. Models and engineering code are preloaded onto the cluster, and large model-internal data transfers are avoided since custom experiment code does not need to be run on the researchers’ own hardware. Critically, unlike traditional HPC and peer-to-peer approaches, NDIF is designed to allow multiple researchers to conduct customized experiments while sharing the underlying preloaded model instances on shared computing hardware.

Safe co-tenancy. NDIF assumes responsibility for ensuring the safe and fair use of its shared model resources via multiple protocols. Interference between user experiments is prevented by the intervention graph’s interleaving mechanism, which enables users to simulate interventions without permanently modifying the model’s original computation graph. Model parameters cannot be directly accessed; instead, users interact with a copy of the parameters. Finally, users can only access models hosted on NDIF if they have been authorized by the model providers. This restriction is naturally enforced, as submitting a request to NDIF through the NNSight API requires downloading the model’s meta version from Hugging Face, where access is managed directly by the model providers.

Compute efficiency. NDIF hosts a selection of preloaded models. For each model, a single shared instance is open to minimize performance difficulties associated with weight-loading and model startup. Researchers can access these models and conduct experiments on them through intervention requests conducted with the NNSight API. The infrastructure implements horizontal scaling and dynamic resource allocation to support multiple user requests and ensure the model deployments are operational and healthy at any given time. The NDIF server facilitates dynamic, bi-directional communication with the client side, enabling users to retrieve and save requested results, and allowing users to minimize additional overhead when running experiments remotely compared to locally. This approach reduces costs for individual researchers, as they no longer need to set up and maintain their

own separate high-performance computing environments. Figure 6 compares NDIF performance to using HPC and Petals.

The code for creating NDIF infrastructure is freely available on GitHub at <https://github.com/ndif-team/ndif>, allowing users to create their own NDIF infrastructure for specialized use cases.

4 PERFORMANCE AND EVALUATION

In this section, we evaluate the performance of NNSight when used remotely, both using exclusive allocations in the traditional high-performance computing setting, and using the shared NDIF inference service. We also compare to the Petals (Borzunov et al., 2023) distributed inference system, which can also reduce the costs of customized inference on huge models. Our evaluation focuses on the time required to load the model weights into memory, as well as the runtime of activation patching, a standard model intervention technique (Vig et al., 2020). For this purpose, we use a single batch of 32 examples from the Indirect Object Identification (IOI) dataset (Wang et al., 2022). We evaluate performance in three different settings: First, we measure the runtime when the models and interventions are executed entirely on a high-performance computing node (HPC). Then, we execute the models and interventions on the NDIF infrastructure, and contrast it against HPC execution times. Finally, we compare NDIF with Petals (Borzunov et al., 2023), and evaluate their relative performance on standard inference tasks as well as on intervention tasks.

High-Performance Computing. We first evaluate NNSight in HPC scenarios by comparing it against other popular libraries designed for intervening on the internal states of PyTorch models, namely TransformerLens (Nanda & Bloom, 2022), pyvene (Wu et al., 2024b), and baukit (Bau, 2022). Our comparison spans models ranging from 1.5 to 8.5 billion parameters. We consider GPT2-XL (Radford et al., 2019), Gemma-7B (Team et al., 2024), and Llama-3.1 8B (AI@Meta, 2024). The experiments were conducted on a single HPC node with four NVIDIA H100 PCIe 82GB GPUs with CUDA version 12.3 and an Intel(R) Xeon(R) Gold 6342 CPU with 24 cores.

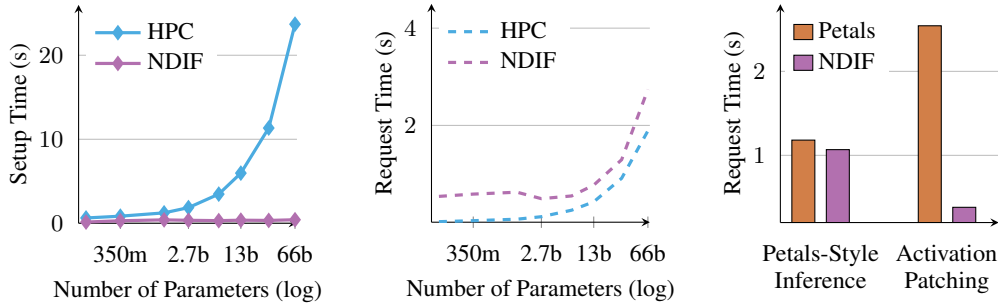
The results are presented in Table 1. We find that the other libraries achieve comparable weight-loading and activation patching times³ to NNSight run without NDIF, for all three models. Therefore we argue that when using exclusive HPC allocations, the choice of library should primarily be guided by factors such as usability, feature set, and compatibility with existing workflows.

Table 1: Runtime comparison of baukit, pyvene, and TransformerLens with NNSight for loading PyTorch models into memory and intervening on their internal states. NNSight achieves comparable performance with other frameworks in both evaluation settings across all three models.

Framework	Setup Time			Activation Patching		
	GPT2-XL	Gemma-7B	Llama-3.1-8B	GPT2-XL	Gemma-7B	Llama-3.1-8B
baukit	3.146 ± 0.006	6.112 ± 0.074	4.529 ± 0.049	0.073 ± 0.001	0.220 ± 0.006	0.352 ± 0.006
pyvene	3.143 ± 0.011	6.263 ± 0.487	5.736 ± 0.717	0.074 ± 0.001	0.222 ± 0.001	0.348 ± 0.006
TransformerLens	8.991 ± 0.024	21.560 ± 0.385	20.625 ± 0.365	0.205 ± 0.027	0.208 ± 0.002	0.332 ± 0.002
NNSight	3.532 ± 0.172	6.351 ± 0.062	5.991 ± 0.192	0.073 ± 0.002	0.227 ± 0.009	0.335 ± 0.010

Remote Execution. We measure the same evaluation settings for HPC versus remotely on an NDIF server. We compare the two setups using the OPT suite of language models (Zhang et al., 2022), ranging from 125 million to 66 billion parameters. For the experiments, we use the same infrastructure as in the comparison against other libraries. The results of this analysis are presented in Figure 6a & 6b. As model size increases, the time required to load the model into memory grows nearly linearly on HPC. In contrast, the time for loading is negligible when using NDIF, since models are preloaded by the service. We also observe that the remote execution on NDIF introduces a roughly constant overhead from communication between the local device and NDIF during activation patching, regardless of model size (see Table 2 in Appendix D). In our setting, remote execution provides runtime improvements for models with 3 billion parameters and more, indicating that remote execution via NDIF becomes increasingly beneficial as parameter size grows.

³There is one exception, as loading the model weights into memory takes about three times as long with TransformerLens as with other libraries. This is likely a result of the preprocessing steps to convert weights into a standardized format across different models.



(a) **HPC vs. NDIF.** We compare the time required to set up the model when NNsight is executed on an HPC node and an NDIF server. For HPC execution, the model must be loaded, so the setup time scales nearly linearly with model size. In contrast, remote execution with NDIF bypasses loading weights.

(b) **HPC vs. NDIF.** We also compare the runtime of activation patching when NNsight is executed on an HPC node, versus using a NNsight locally to remotely access an NDIF server. We observe that the remote execution introduces a roughly constant overhead from communication between the local device and NDIF.

(c) **Petals vs. NDIF.** We compare NDIF against Petals, an open-source remote inference framework. We find that the libraries demonstrate comparable performance for standard remote inference, but that NDIF significantly outperforms Petals for remote interventions.

Figure 6: Performance evaluation of NNsight and NDIF. In all experiments, the sample size is $n = 128$. Plot marker widths are comparable to the 95% confidence intervals.

Distributed inference. We also compare NDIF against Petals, an open-source service for distributed model inference (Borzunov et al., 2023). Petals enables inference on large language models across shared resources by transmitting intermediate hidden states. To ensure a fair comparison, we deployed private instances of both Petals and NDIF on a server with a single NVIDIA RTX A6000 49 GB GPUs with CUDA version 12.5 and an AMD Ryzen 9 5900X CPU with 12 cores. The communication between these instances took place via a network with a bandwidth of about 60 MB/s. As Petals was primarily designed for standard remote inference, we first compare the two services in this context. With Petals, the client transmits token embeddings to the server and receives final hidden states, which can then be mapped to token probabilities. In contrast, NDIF sends the input data and the intervention graph to the server. While NDIF could simply return the next token prediction, we opted to return the final hidden states to the client for a fair comparison. The results are presented in Figure 6c. In this scenario, both frameworks demonstrate comparable performance.

Petals also support certain types of inference-time interventions on model internals. Therefore, we also compare NDIF and Petals in executing activation patching. Petals allows returning intermediate hidden states, which can be used for interventions on the model’s residual stream. However, as Petals does not support server-side interventions, modifications to the internal state must be performed locally on the client device. This process involves receiving hidden states at a specific layer, performing local modifications, and then sending the modified hidden states back to the server. In contrast, NDIF supports server-side interventions and metric computations. This allows us to avoid transmitting hidden states between client and server by computing patching metrics (e.g., logit difference) on the server and returning only those. The results are presented in Figure 6c. In this scenario, we find that NDIF significantly outperforms Petals.

5 RELATED WORK

Research model hosting frameworks. The most similar previous works are Garçon (Elhage et al., 2021) and Petals (Borzunov et al., 2023). Garçon is a proprietary internal research system at Anthropic that supports remote inspection and customization of large models hosted on a remote cluster. Unlike the computation-graph approach of NNsight, Garçon operates by allowing researchers to hook models via arbitrary-code execution. As a result, co-tenancy is unsafe, so unlike the NDIF server that shares model instances between many users, Garçon requires each user to allocate their own model instance, which is computationally expensive. The Petals system has similar goals, but instead of distributing experiment code, it distributes foundation-model computation across user-contributed nodes in a BitTorrent-style swarm. Petals achieves co-tenancy by leaving researcher-defined computations on

the user’s own system. Unlike Petals, NNsight can avoid costly model-internal data transfers by executing computation graphs on NDIF servers.

Several other solutions for sharing hosted model resources have been developed, including VLLM (Kwon et al., 2023), Huggingface TGI (Dehaene et al., 2024) and Nvidia TensorRT-LLM (Nvidia, 2024). These are systems for LLM inference acceleration that support large-scale multiuser model sharing. However, unlike NDIF, these systems only provide black-box API access to models, and do not permit model inspection or modification, which limits their utility for interpretability research. Also related to our work are systems such as S-LoRA (Sheng et al., 2024), dLoRA (Wu et al., 2024a), and Punica (Chen et al., 2024), which excel at efficient serving of many pretrained LoRA adapters in parallel, and systems such as FusionAI and FusionLLM (Tang et al., 2023; 2024) which support decentralized training. While these LoRA-focused systems enable one form of model modification, NNsight and NDIF provide finer-grained control that allows for a wider range of model interventions.

Frameworks for model internals. Numerous efforts have been made to create robust tools for exploring and manipulating model internals. These tools are all designed to support *local* experiments, executed on compute resources controlled by the researcher conducting them. TransformerLens (Nanda & Bloom, 2022) is designed to facilitate the exploration of GPT-style decoder-only language models. It allows users to apply custom functions to outputs of specific model modules. Unlike NNsight, which operates on arbitrary PyTorch networks, TransformerLens is limited to transformer-based language models. Pyvene (Wu et al., 2024b) supports configurable intervention schemes on PyTorch modules, decorating a PyTorch module with hooks that allow activations to be collected and overwritten. Pyvene operates at a higher level of abstraction, and can be configured as a layer over NNsight. Baukit (Bau, 2022) provides a range of utilities that simplify tracing and editing activations for local models. However, unlike NNsight’s intervention graph, it lacks an intermediate representation that would allow for interaction with larger, remotely-hosted models. Penzai (Johnson, 2024) is an open-source functional library which provides a modular system that allows for introspection, visualization, and manipulation of neural network computation graphs; Penzai works within the JAX framework, in contrast to NNsight which works with PyTorch models.

6 DISCUSSION

The growing scale of highly-capable AI systems has presented our field with an access challenge: The most capable systems are increasingly costly to deploy, and that has meant that many of the most interesting and important research targets have become difficult for scientists to study in depth. In this paper we have proposed an architecture for reducing barriers to research by defining a technical approach for separating experiment definition from network deployment. Together, the intervention graph architecture, the NNsight API, and the NDIF service enable researchers to perform complex experiments and explore neural network internals at a scale previously limited by computational, engineering, and financial constraints.

While our approach addresses technical issues for conducting research on very large open-weight models, a key limitation is that it does not provide access to closed-parameter, proprietary models like GPT-4 or Claude. However, proprietary vendors often have special requirements. For instance, they may wish to maintain secrecy and security of model parameters for business or safety reasons. Our system is designed with these concerns in mind, and it is technically feasible for companies to support under these requirements. We encourage commercial providers to consider adopting our approach as part of their products to enable transparency and facilitate future scientific progress at the frontiers of AI.

NDIF and NNsight define a way of doing research that acknowledges the important role of community resources. Scientists in other fields have opened up new research vistas by pooling investments in shared research instruments, databases, and facilities. By enabling the AI research community to similarly pool its resources for large-scale inference, we can open a pathway for conducting research on AI at scales that would be unattainable for individual research groups.

7 ETHICS STATEMENT

Our work is intended to democratize access to large neural networks by providing tools that make it easier for researchers to audit, understand, and interpret large AI models. We caution that such transparency may also enable abuse of large models, for example exploiting model vulnerabilities. Therefore, we strongly urge the responsible and ethical use, deployment, and monitoring of our tools, while emphasizing the importance of continued research to improve model interpretation.

8 REPRODUCIBILITY STATEMENT

To promote transparency and ensure the reproducibility of our results, the complete NNsight and NDIF frameworks, along with comprehensive documentation and all relevant experiment code, are available at <https://github.com/ndif-team/nnsight> and <https://github.com/ndif-team/ndif>, respectively. We hope this will allow researchers and practitioners to build upon our work.

9 ACKNOWLEDGEMENTS

The National Deep Inference Fabric (NDIF) and NNsight are supported by the U.S. National Science Foundation Award IIS-2408455. We are also grateful for the support of Open Philanthropy (ET, KP, SM, AS, NP, DB), Manifund Reagents (CR), and the Zuckerman Postdoctoral Fellowship (AM). We thank Brett Bode, William Gropp, Greg Bauer, and Volodymyr Kindratenko at NCSA for their help working with DeltaAI, which provides the primary computing infrastructure for NDIF, and we thank Thea Shaheen for invaluable assistance with infrastructure for the project at Northeastern. We thank the New England Research Cloud (NERC) for hosting the web frontend for NDIF. We also thank Callum McDougall for developing the NNsight tutorial in the ARENA 3.0 materials. We thank Clément Dumas and Gonçalo Paulo for valuable feedback as early research adopters, Zhengxuan Wu for implementing pyvene on NNsight, and Zhongbo Zhu for valuable advice on serving infrastructure. We also acknowledge the many early testers for the Llama 3.1 405B pilot program. Finally, we are grateful for the ongoing feedback and engagement of our community of users on Discord.

REFERENCES

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: a system for Large-Scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pp. 265–283, 2016.
- AI@Meta. Llama 3 model card, 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- Rami Al-Rfou, Guillaume Alain, Amjad Almahairi, Christof Angermüller, Dzmitry Bahdanau, Nicolas Ballas, Frédéric Bastien, Justin Bayer, Anatoly Belikov, Alexander Belopolsky, Yoshua Bengio, Arnaud Bergeron, James Bergstra, Valentin Bisson, Josh Blecher Snyder, Nicolas Bouchard, Nicolas Boulanger-Lewandowski, Xavier Bouthillier, Alexandre de Brébisson, Olivier Breuleux, Pierre Luc Carrier, Kyunghyun Cho, Jan Chorowski, Paul Francis Christiano, Tim Cooijmans, Marc-Alexandre Côté, Myriam Côté, Aaron C. Courville, Yann Dauphin, Olivier Delalleau, Julien Demouth, Guillaume Desjardins, Sander Dieleman, Laurent Dinh, Mélanie Ducoffe, Vincent Dumoulin, Samira Ebrahimi Kahou, D. Erhan, Ziyi Fan, Orhan Firat, Mathieu Germain, Xavier Glorot, Ian J. Goodfellow, M. Graham, Çağlar Gülçehre, Philippe Hamel, Iban Harlouchet, Jean-Philippe Heng, Balázs Hidasi, Sina Honari, Arjun Jain, Sébastien Jean, Kai Jia, Mikhail Korobov, Vivek Kulkarni, Alex Lamb, Pascal Lamblin, Eric Larsen, César Laurent, Sea Sun Lee, Simon Lefrançois, Simon Lemieux, Nicholas Léonard, Zhouhan Lin, Jesse A. Livezey, Cory Lorenz, Jeremiah Lowin, Qianli Ma, Pierre-Antoine Manzagol, Olivier Mastropietro, Robert T. McGibbon, Roland Memisevic, Bart van Merriënboer, Vincent Michalski, Mehdi Mirza, Alberto Orlandi, Christopher Joseph Pal, Razvan Pascanu, Mohammad Pezeshki, Colin Raffel, Daniel Renshaw, Matthew Rocklin, Adriana Romero, Markus Roth, Peter Sadowski, John Salvatier, François Savard, Jan Schlüter, John Schulman, Gabriel Schwartz, Iulian Serban, Dmitriy Serdyuk, Samira Shabanian, Étienne Simon, Sigurd Spieckermann, S. Ramana Subramanyam, Jakub Sygnowski, Jérémie Tanguay, Gijs van

- Tulder, Joseph P. Turian, Sebastian Urban, Pascal Vincent, Francesco Visin, Harm de Vries, David Warde-Farley, Dustin J. Webb, Matthew Willson, Kelvin Xu, Lijun Xue, Li Yao, Saizheng Zhang, and Ying Zhang. Theano: A python framework for fast computation of mathematical expressions. *ArXiv*, abs/1605.02688, 2016. URL <https://api.semanticscholar.org/CorpusID:8993325>.
- Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented scale. *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–15, 2022. URL <https://api.semanticscholar.org/CorpusID:250243681>.
- Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Gradient-based attribution methods. *Explainable AI: Interpreting, explaining and visualizing deep learning*, pp. 169–191, 2019.
- Anthropic. The claude 3 model family: Opus, sonnet, haiku, 2024. URL <https://api.semanticscholar.org/CorpusID:268232499>.
- Jasmijn Bastings, Sebastian Ebert, Polina Zablotskaia, Anders Sandholm, and Katja Filippova. “will you find these shortcuts?” a protocol for evaluating the faithfulness of input salience methods for text classification. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 976–991, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.64. URL <https://aclanthology.org/2022.emnlp-main.64>.
- David Bau. Baukit, 2022. URL <https://github.com/davidbau/baukit>.
- David Bau, Bolei Zhou, Aditya Khosla, Aude Oliva, and Antonio Torralba. Network dissection: Quantifying interpretability of deep visual representations. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6541–6549, July 2017.
- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, Usven Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. Pythia: A suite for analyzing large language models across training and scaling. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. In Danushka Bollegala, Ruihong Huang, and Alan Ritter (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 558–568, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-demo.54. URL <https://aclanthology.org/2023.acl-demo.54>.
- Léon Bottou and Patrick Gallinari. A framework for the cooperation of learning algorithms. *Advances in neural information processing systems*, 3, 1990.
- Léon Bottou and Yann Le Cun. Sn: A simulator for connectionist models. In *Proceedings of NeuroNimes 88*, pp. 371–382, Nimes, France, 1988. URL <http://leon.bottou.org/papers/bottou-lecun-88>.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

- Benjamin S Bucknall and Robert F Trager. Structured access for third-party research on frontier ai models: Investigating researcher’s model access requirements, 2023. URL https://cdn.governance.ai/Structured_Access_for_Third-Party_Research.pdf.
- Stephen Casper, Carson Ezell, Charlotte Siegmman, Noam Kolt, Taylor Lynn Curtis, Benjamin Bucknall, Andreas Haupt, Kevin Wei, J       Scheurer, Marius Hobbhahn, et al. Black-box access is insufficient for rigorous ai audits. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency*, pp. 2254–2272, 2024.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant lora serving. In P. Gibbons, G. Pekhimenko, and C. De Sa (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 1–13, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/054de805fcceb78a201f5e9d53c85908-Paper-Conference.pdf.
- Olivier Dehaene, Nicolas Patry, David Richard Holtz, Dani     de Kok, and contributors. HuggingFace text generation interface. URL <https://github.com/huggingface/text-generation-inference>, 2024.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=OUIFPHEgJU>.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Lauren Rantala-Yearly, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur   elebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gouget, Virginie Do, Vish Voleti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyan Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen Tan, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe

Papakipos, Aaditya Singh, Aaron Grattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardt, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzmán, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelen, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Maria Tsimpoukelli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natasha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Rohan Maheswari, Russ Howes, Ruty Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vitor Albiero, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaofang Wang, Xiaojuan Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.

Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. Garcon. URL <https://transformer-circuits.pub/2021/garcon/index.html>, 2021.

Epoch AI. Data on notable ai models, 2024. URL <https://epochai.org/data/notable-ai-models>. Last Accessed: 2024-09-30.

Javier Ferrando, Gabriele Sarti, Arianna Bisazza, and Marta R Costa-jussà. A primer on the inner workings of transformer-based language models. *arXiv preprint arXiv:2405.00208*, 2024.

Gemini Team, Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy P. Lillcrap, Jean-Baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, Ioannis Antonoglou, Rohan Anil, Sebastian Borgeaud, Andrew M. Dai, Katie Millican, Ethan Dyer, Mia Glaese, Thibault Sottiaux, Benjamin Lee, Fabio Viola, Malcolm Reynolds, Yuanzhong Xu, James Molloy, Jilin Chen, Michael Isard, Paul Barham, Tom Hennigan, Ross McIlroy, Melvin Johnson, Johan Schalkwyk, Eli Collins, Eliza Rutherford, Erica Moreira, Kareem W. Ayoub, Megha Goel, Clemens Meyer, Gregory Thornton, Zhen Yang, Henryk Michalewski, Zaheer Abbas, Nathan Schucher, Ankesh Anand, Richard Ives, James Keeling, Karel Lenc, Salem Haykal, Siamak Shakeri, Pranav Shyam, Aakanksha Chowdhery, Roman Ring, Stephen Spencer, Eren Sezener, Luke Vilnis, Oscar Chang, Nobuyuki Morioka, George Tucker, Ce Zheng, Oliver Woodman, Nithya Attaluri, Tomás Kociský, Evgenii Eltyshev, Xi Chen, Timothy Chung, Vittorio Selo, Siddhartha Brahma, Petko Georgiev, Ambrose Slone, Zhenkai Zhu, James Lottes, Siyuan Qiao, Ben Caine, Sebastian Riedel, Alex Tomala, Martin Chadwick, J Christopher Love, Peter Choy, Sid Mittal, Neil Houlsby, Yunhao Tang, Matthew Lamm, Libin Bai, Qiao Zhang, Luheng He, Yong Cheng, Peter Humphreys, Yujia Li, Sergey Brin, Albin Cassirer, Ying-Qi Miao, Lukás Zilka, Taylor Tobin, Kelvin Xu, Lev Proleev, Daniel Sohn, Alberto Magni, Lisa Anne Hendricks, Isabel Gao, Santiago Ontan’on, Oskar Bunyan, Nathan Byrd, Abhanshu Sharma, Biao Zhang, Mario Pinto, Rishika Sinha, Harsh Mehta, Dawei Jia, Sergi Caelles, Albert Webson, Alex Morris, Becca Roelofs, Yifan Ding, Robin Strudel, Xuehan Xiong, Marvin Ritter, Mostafa Dehghani, Rahma Chaabouni, Abhijit Karmarkar, Guangda Lai, Fabian Mentzer, Bibo Xu, YaGuang Li, Yujing Zhang, Tom Le Paine, Alex Goldin, Behnam Neyshabur, Kate Baumli, Anselm Levskaya, Michael Laskin, Wenhao Jia, Jack W. Rae, Kefan Xiao, Antoine He, Skye Giordano, Lakshman Yagati, Jean-Baptiste Lepiau, Paul Natsev, Sanjay Ganapathy, Fangyu Liu, Danilo Martins, Nanxin Chen, Yunhan Xu, Megan Barnes, Rhys May, Arpi Vezar, Junhyuk Oh, Ken Franko, Sophie Bridgers, Ruizhe Zhao, Boxi Wu, Basil Mustafa, Sean Sechrist, Emilio Parisotto, Thanumalayan Sankaranarayana Pillai, Chris Larkin, Chenjie Gu, Christina Sorokin, Maxim Krikun, Alexey Guseynov, Jessica Landon, Romina Datta, Alexander Pritzel, Phoebe Thacker, Fan Yang, Kevin Hui, A.E. Hauth, Chih-Kuan Yeh, David Barker, Justin Mao-Jones, Sophia Austin, Hannah Sheahan, Parker Schuh, James Svensson, Rohan Jain, Vinay Venkatesh Ramasesh, Anton Briukhov, Da-Woon Chung, Tamara von Glehn, Christina Butterfield, Priya Jhakra, Matt Wiethoff, Justin Frye, Jordan Grimstad, Beer Changpinyo, Charline Le Lan, Anna Bortsova, Yonghui Wu, Paul Voigtlaender, Tara N. Sainath, Charlotte Smith, Will Hawkins, Kris Cao, James Besley, Srivatsan Srinivasan, Mark Omernick, Colin Gaffney, Gabriela de Castro Surita, Ryan Burnell, Bogdan Damoc, Junwhan Ahn, Andrew Brock, Mantas Pajarskas, Anastasia Petrushkina, Seb Noury, Lorenzo Blanco, Kevin Swersky, Arun Ahuja, Thi Avrahami, Vedant Misra, Raoul de Liedekerke, Mariko Iinuma, Alex Polozov, Sarah York, George van den Driessche, Paul Michel, Justin Chiu, Rory Blevins, Zach Gleicher, Adrià Recasens, Alban Rustemi, Elena Gribovskaya, Aurko Roy, Wiktor Gworek, Sébastien M. R. Arnold, Lisa Lee, James Lee-Thorp, Marcello Maggioni, Demis Hassabis, Koray Kavukcuoglu, Jeffrey Dean, and Oriol Vinyals. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *ArXiv*, abs/2403.05530, 2024. URL <https://api.semanticscholar.org/CorpusID:268297180>.

Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual associations in auto-regressive language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12216–12235, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

J. Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *ArXiv*, abs/2106.09685, 2021. URL <https://api.semanticscholar.org/CorpusID:235458009>.

- Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=F76bwRSLeK>.
- Huggingface. Huggingface open llm leaderboard archive, 2024. URL https://huggingface.co/spaces/open-llm-leaderboard-old/open_llm_leaderboard. Last Accessed: 2024-09-30.
- Albert Qiaochu Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L’elio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mistral 7b. *ArXiv*, abs/2310.06825, 2023. URL <https://api.semanticscholar.org/CorpusID:263830494>.
- Daniel D. Johnson. Penzai + Treescop: A toolkit for interpreting, visualizing, and editing models as data. *ICML 2024 Workshop on Mechanistic Interpretability*, 2024. URL <https://openreview.net/forum?id=KVSgEXrMDH>.
- János Kramár, Tom Lieberum, Rohin Shah, and Neel Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3045–3059, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.243. URL <https://aclanthology.org/2021.emnlp-main.243>.
- Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in Neural Information Processing Systems*, 36, 2024.
- Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models, 2024. URL <https://arxiv.org/abs/2403.19647>.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. In *Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=h6WAS6eE4>.
- Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications, 2018. URL <https://arxiv.org/abs/1712.05889>.
- Neel Nanda and Joseph Bloom. Transformerlens. <https://github.com/TransformerLensOrg/TransformerLens>, 2022.
- Nvidia. TensorRT-LLM. URL <https://github.com/NVIDIA/TensorRT-LLM>, 2024.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mo Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Benjamin Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah

Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Sim'on Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Raphael Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Lukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Hendrik Kirchner, Jamie Ryan Kiros, Matthew Knight, Daniel Kokotajlo, Lukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kopic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Adeola Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel P. Mossing, Tong Mu, Mira Murati, Oleg Murk, David M'ely, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Ouyang Long, Cullen O'Keefe, Jakub W. Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alexandre Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Pondé de Oliveira Pinto, Michael Pokorný, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario D. Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin D. Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas A. Tezak, Madeleine Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cer'on Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll L. Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. URL <https://api.semanticscholar.org/CorpusID:257532815>.

PaperswithCode. Multi-task language understanding on mmlu, 2024. URL <https://paperswithcode.com/sota/multi-task-language-understanding-on-mmlu>. Last Accessed: 2024-09-30.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

Roma Patel and Ellie Pavlick. Mapping language models to grounded conceptual spaces. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=gJcEM8sxHK>.

Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hengnigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri,

Saffron Huang, Jonathan Uesato, John F. J. Mellor, Irina Higgins, Antonia Creswell, Nathan McAleese, Amy Wu, Erich Elsen, Siddhant M. Jayakumar, Elena Buchatskaya, David Buden, Esme Sutherland, Karen Simonyan, Michela Paganini, L. Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, N. K. Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Tobias Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d’Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew G. Johnson, Blake A. Hechtman, Laura Weidinger, Iason Gabriel, William S. Isaac, Edward Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem W. Ayoub, Jeff Stanway, L. L. Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. Scaling language models: Methods, analysis & insights from training gopher. *ArXiv*, abs/2112.11446, 2021. URL <https://api.semanticscholar.org/CorpusID:245353475>.

Ray Team. Ray, 2022. URL <https://github.com/ray-project/ray>.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagn’e, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Pawan Sasanka Ammanamanchi, Thomas Wang, Benoît Sagot, Niklas Muennighoff, Albert Villanova del Moral, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, Iz Beltagy, Huu Nguyen, Lucile Saulnier, Samson Tan, Pedro Ortiz Suarez, Victor Sanh, Hugo Laurenceon, Yacine Jernite, Julien Launay, Margaret Mitchell, Colin Raffel, Aaron Gokaslan, Adi Simhi, Aitor Soroa Etxabe, Alham Fikri Aji, Amit Alfassy, Anna Rogers, Ariel Kreisberg Nitzav, Canwen Xu, Chenghao Mou, Chris C. Emezue, Christopher Klamm, Colin Leong, Daniel Alexander van Strien, David Ifeoluwa Adelani, Dragomir R. Radev, Eduardo Gonz’alez Ponferrada, Efrat Levkovizh, Ethan Kim, Eyal Natan, Francesco De Toni, Gérard Dupont, Germán Kruszewski, Giada Pistilli, Hady ElSahar, Hamza Benyamina, Hieu Trung Tran, Ian Yu, Idris Abdulmumin, Isaac Johnson, Itziar Gonzalez-Dios, Javier de la Rosa, Jenny Chim, Jesse Dodge, Jian Zhu, Jonathan Chang, Jorg Frohberg, Josephine Tobing, Joydeep Bhattacharjee, Khalid Almubarak, Kimbo Chen, Kyle Lo, Leandro von Werra, Leon Weber, Long Phan, Loubna Ben Allal, Ludovic Tanguy, Manan Dey, Manuel Romero Muñoz, Maraim Masoud, María Grandury, Mario vSavsko, Max Huang, Maximin Coavoux, Mayank Singh, Mike Tian-Jian Jiang, Minh Chien Vu, Mohammad A. Jauhar, Mustafa Ghaleb, Nishant Subramani, Nora Kassner, Nurulaqilla Khamis, Olivier Nguyen, Omar Espejel, Ona de Gibert, Paulo Villegas, Peter Henderson, Pierre Colombo, Priscilla Amuok, Quentin Lhoest, Rheza Harliman, Rishi Bommasani, Roberto L’opez, Rui Ribeiro, Salomey Osei, Sampo Pyysalo, Sebastian Nagel, Shamik Bose, Shamsuddeen Hassan Muhammad, Shanya Sharma, S. Longpre, Somaieh Nikpoor, S. Silberberg, Suhas Pai, Sydney Zink, Tiago Timponi Torrent, Timo Schick, Tristan Thrush, Valentin Danchev, Vassilina Nikoulina, Veronika Laippala, Violette Lepercq, Vrinda Prabhu, Zaid Alyafeai, Zeerak Talat, Arun Raja, Benjamin Heinzerling, Chenglei Si, Elizabeth Salesky, Sabrina J. Mielke, Wilson Y. Lee, Abheesh Sharma, Andrea Santilli, Antoine Chaffin, Arnaud Stiegler, Debajyoti Datta, Eliza Szczechla, Gunjan Chhablani, Han Wang, Harshit Pandey, Hendrik Strobelt, Jason Alan Fries, Jos Rozen, Leo Gao, Lintang Sutawika, M Saiful Bari, Maged S. Al-Shaibani, Matteo Manica, Nihal V. Nayak, Ryan Teehan, Samuel Albanie, Sheng Shen, Srulik Ben-David, Stephen H. Bach, Taewoon Kim, Tali Bers, Thibault Févry, Trishala Neeraj, Urmish Thakker, Vikas Raunak, Xiang Tang, Zheng-Xin Yong, Zhiqing Sun, Shaked Brody, Y Uri, Hadar Tojarieh, Adam Roberts, Hyung Won Chung, Jaesung Tae, Jason Phang, Ofir Press, Conglong Li, Deepak Narayanan, Hatim Bourfoune, Jared Casper, Jeff Rasley, Max Ryabinin, Mayank Mishra, Minjia Zhang, Mohammad Shoeybi, Myriam Peyrounette, Nicolas Patry, Nouamane Tazi, Omar Sanseviero, Patrick von Platen, Pierre Cornette, Pierre Francois Lavall’ee, Rémi Lacroix, Samyam Rajbhandari, Sanchit Gandhi, Shaden Smith, Stéphane Requena, Suraj Patil, Tim Dettmers, Ahmed Baruwa, Amanpreet Singh, Anastasia Cheveleva, Anne-Laure Ligozat, Arjun Subramonian, Aur’elie N’ev’eol, Charles Lovering, Daniel H Garrette, Deepak R. Tunuguntla, Ehud Reiter, Ekaterina Taktasheva, Ekaterina Voloshina, Eli Bogdanov, Genta Indra Winata, Hailey Schoelkopf, Jan-Christoph Kalo, Jekaterina Novikova, Jessica Zosa Forde, Xiangru Tang, Jungo Kasai, Ken Kawamura, Liam Hazan, Marine Carpuat, Miruna Clinciu, Najoung Kim, Newton Cheng, Oleg Serikov, Omer Antverg, Oskar van der Wal, Rui Zhang, Ruochen Zhang, Sebastian Gehrmann, Shachar Mirkin, S. Osher Pais, Tatiana Shavrina, Thomas Scialom, Tian Yun, Tomasz Limisiewicz, Verena Rieser, Vitaly Protasov, Vladislav Mikhailov, Yada Pruksachatkun, Yonatan Belinkov, Zachary Bamberger, Zdenek Kasner, Zdeněk Kasner, Amanda Pestana, Amir

- Feizpour, Ammar Khan, Amy Faranak, Ananda Santa Rosa Santos, Anthony Hevia, Antigona Undreaaj, Arash Aghagol, Arezoo Abdollahi, Aycha Tammour, Azadeh HajiHosseini, Bahareh Behroozi, Benjamin Ayoade Ajibade, Bharat Kumar Saxena, Carlos Muñoz Ferrandis, Danish Contractor, David M. Lansky, Davis David, Douwe Kiela, Duong Anh Nguyen, Edward Tan, Emi Baylor, Ezinwanne Ozoani, Fatim Tahirah Mirza, Frankline Ononiwu, Habib Rezanejad, H.A. Jones, Indrani Bhattacharya, Irene Solaiman, Irina Sedenko, Isar Nejadgholi, Jan Passmore, Joshua Seltzer, Julio Bonis Sanz, Karen Fort, Livia Dutra, Mairon Samagaio, Maraim Elbadri, Margot Mieskes, Marissa Gerchick, Martha Akinlolu, Michael McKenna, Mike Qiu, Muhammed Ghauri, Mykola Burynek, Nafis Abrar, Nazneen Rajani, Nour Elkott, Nourhan Fahmy, Olanrewaju Samuel, Ran An, R. P. Kromann, Ryan Hao, Samira Alizadeh, Sarmad Shubber, Silas L. Wang, Sourav Roy, Sylvain Viguier, Thanh-Cong Le, Tobi Oyeade, Trieu Nguyen Hai Le, Yoyo Yang, Zach Nguyen, Abhinav Ramesh Kashyap, Alfredo Palasciano, Alison Callahan, Anima Shukla, Antonio Miranda-Escalada, Ayush Kumar Singh, Benjamin Beilharz, Bo Wang, Caio Matheus Fonseca de Brito, Chenxi Zhou, Chirag Jain, Chuxin Xu, Clémentine Fourier, Daniel Le'on Perin'an, Daniel Molano, Dian Yu, Enrique Manjavacas, Fabio Barth, Florian Fuhrmann, Gabriel Altay, Giyaseddin Bayrak, Gully Burns, Helena U. Vrabec, Iman I.B. Bello, Isha Dash, Ji Soo Kang, John Giorgi, Jonas Golde, Jose David Posada, Karthi Sivaraman, Lokesh Bulchandani, Lu Liu, Luisa Shinzato, Madeleine Hahn de Bykhovetz, Maiko Takeuchi, Marc Pàmies, María Andrea Castillo, Marianna Nezhurina, Mario Sanger, Matthias Samwald, Michael Cullan, Michael Weinberg, M Wolf, Mina Mihaljcic, Minna Liu, Moritz Freidank, Myungsun Kang, Natasha Seelam, Nathan Dahlberg, Nicholas Michio Broad, Nikolaus Muellner, Pascale Fung, Patricia Haller, R. Chandrasekhar, Renata Eisenberg, Robert Martin, Rodrigo Canalli, Rosaline Su, Ruisi Su, Samuel Cahyawijaya, Samuele Garda, Shlok S Deshmukh, Shubhanshu Mishra, Sid Kiblawi, Simon Ott, Sinee Sang-aaroonsiri, Srishti Kumar, Stefan Schweter, Sushil Pratap Bharati, Tanmay Laud, Théo Gigant, Tomoya Kainuma, Wojciech Kusa, Yanis Labrak, Yashasvi Bajaj, Y. Venkatraman, Yifan Xu, Ying Xu, Yu Xu, Zhee Xiao Tan, Zhongli Xie, Zifan Ye, Mathilde Bras, Younes Belkada, and Thomas Wolf. Bloom: A 176b-parameter open-access multilingual language model. *ArXiv*, abs/2211.05100, 2022. URL <https://api.semanticscholar.org/CorpusID:253420279>.
- Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=ITw9edRD1D>.
- Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph Gonzalez, and Ion Stoica. Slora: Scalable serving of thousands of lora adapters. In P. Gibbons, G. Pekhimenko, and C. De Sa (eds.), *Proceedings of Machine Learning and Systems*, volume 6, pp. 296–311, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/906419cd502575b617cc489a1a696a67-Paper-Conference.pdf.
- Toby Shevlane. Structured access: an emerging paradigm for safe ai deployment, 2022. URL <https://arxiv.org/abs/2201.05159>.
- Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 3319–3328. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/sundararajan17a.html>.
- Zhenheng Tang, Yuxin Wang, Xin He, Longteng Zhang, Xinglin Pan, Qiang Wang, Rongfei Zeng, Kaiyong Zhao, Shaohuai Shi, Bingsheng He, et al. Fusionai: Decentralized training and deploying llms with massive consumer-level gpus. *arXiv preprint arXiv:2309.01172*, 2023.
- Zhenheng Tang, Xueze Kang, Yiming Yin, Xinglin Pan, Yuxin Wang, Xin He, Qiang Wang, Rongfei Zeng, Kaiyong Zhao, Shaohuai Shi, et al. Fusionllm: A decentralized llm training system on geo-distributed gpus with adaptive compression. *arXiv preprint arXiv:2410.12707*, 2024.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Husenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikula, Mateo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Hennigan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open models based on gemini research and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.

Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Daniel M. Bikel, Lukas Blecher, Cristian Cantón Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony S. Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel M. Kloumann, A. V. Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, R. Subramanian, Xia Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zhengxu Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. *ArXiv*, abs/2307.09288, 2023. URL <https://api.semanticscholar.org/CorpusID:259950998>.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 12388–12401. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/92650b2e92217715fe312e6fa7b90d82-Paper.pdf.

Kevin Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022. ISSN 2835-8856. URL <https://openreview.net/forum?id=yzkSU5zdwD>. Survey Certification.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von

- Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. Transformers: State-of-the-art natural language processing. In Qun Liu and David Schlangen (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-demos.6. URL <https://aclanthology.org/2020.emnlp-demos.6>.
- Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. {dLoRA}: Dynamically orchestrating requests and adapters for {LoRA}{LLM} serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 911–927, 2024a.
- Zhengxuan Wu, Atticus Geiger, Thomas Icard, Christopher Potts, and Noah Goodman. Interpretability at scale: Identifying causal mechanisms in alpaca. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 78205–78226. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/f6a8b109d4d4fd64c75e94aaf85d9697-Paper-Conference.pdf.
- Zhengxuan Wu, Atticus Geiger, Aryaman Arora, Jing Huang, Zheng Wang, Noah Goodman, Christopher Manning, and Christopher Potts. pyvene: A library for understanding and improving PyTorch models via interventions. In Kai-Wei Chang, Annie Lee, and Nazneen Rajani (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: System Demonstrations)*, pp. 158–165, Mexico City, Mexico, June 2024b. Association for Computational Linguistics. URL <https://aclanthology.org/2024.naacl-demo.16>.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024.
- Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Hf17y6u9BC>.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. Opt: Open pre-trained transformer language models, 2022. URL <https://arxiv.org/abs/2205.01068>.

A RESEARCH SURVEY DETAILS

In this section, we provide details on the data collection process for the results reported in Section 2, and report additional findings related to model parameter size.

Data Curation. We curated a list of 184 total papers that study the internals of open-weight transformer models, taken from the citations of a recent survey of interpretability research (Ferrando et al., 2024). Starting from the initial list of 411 citations, we excluded any paper that met one or more of the following criteria:

- Paper is a survey paper, tutorial, library report, model card, or performs no model experiments.
- Paper only performs experiments on closed-weight models, custom models, or non-transformer architectures.
- Paper was published before 2019.

We identified the largest model (by parameter count) studied in each paper and gathered data related to its MMLU performance (Hendrycks et al., 2021) from the HuggingFace OpenLLM Leaderboard Archive (Huggingface, 2024). We used the Papers with Code MMLU Leaderboard (PaperswithCode, 2024) to supplement MMLU performance data when models did not have a score on the Huggingface leaderboard. When a model had a score from both sources, we averaged its benchmark results together. A few models did not have reported MMLU results from either source (primarily BERT-style models). For these models, we interpolated MMLU results using nearest neighbor approximation based on parameter size. Papers with Code provides an MMLU performance reference for closed-weight models.

Figure 7: Evolution of the size gap between open-weight models used in research and publicly released models from 2019 to 2024. Blue boxes represent the distribution of model sizes used in research papers, while pink boxes show publicly released models. The dashed gold line connects median model sizes for each group, with the ratio between medians displayed. Outliers are shown as individual points. The growing disparity (from 2.7x in 2019-2020 to 10.3x in 2024) suggests that interpretability research is increasingly lagging behind the capabilities of state-of-the-art models available to the public.

Comparing Model Parameter Size. In Figure 7, we show there is a growing disparity between the size of models that interpretability researchers are studying and the size of models that are publicly released each year, a similar trend to the one discussed in §2 regarding MMLU. Reference data on parameter sizes of publicly available models is taken from Epoch AI (2024), restricting our analysis to language models released after 2019 that have openly downloadable parameters. To ensure reproducibility and transparency, all source code and data used for the analyses described in this section are available in the supplementary materials accompanying this paper.

B SYSTEM DESIGN

In this section, we provide additional details describing the implementation and design of NNSight and NDIF.

B.1 NNSIGHT DESIGN

Wrapped PyTorch modules To support the tracing context, the NNSight library wraps PyTorch modules in an NNSight object instance. This wrapper is responsible for providing access to its sub-modules’ intermediate deferred inputs and outputs. Upon initialization, the NNSight object creates an Envoy object for each sub-module, forming a tree-like structure mirroring that of the original PyTorch module tree. Each Envoy is responsible for managing and recording operations on future inputs and outputs for its underlying module. It achieves this by exposing attributes like `.input` and `.output` which, when accessed, trace a deferred operation to intervene at the corresponding module’s input or output during model execution. This attribute access returns a Proxy object of that deferred operation, representing a future intermediate value. Any operation performed on the resulting Proxy object creates a new deferred operation, and therefore a new Proxy. As all future Proxys originate from operations performed on these root `.input` and `.output` attributes, it makes them the entry-point into the tracing context functionality.

```
1 from nnsight import NNSight
2 model = NNSight(model)
3 input = tensor([.46, 1.56, -0.3, .98, -0.5])
4 with model.trace(input):
5     model.submodule.output[:2] = 0
6     hidden_state = model.submodule.output.save()
```

Code Example 1: Basic usage of the NNSight tracing context and the NNSight data type. `model` in this example has a single submodule named “submodule” and accepts a Tensor of length five. On line two, the PyTorch model is wrapped with NNSight. On line four, the tracing context is created and entered by calling `with model.trace(...)` given some input. On line five, the Envoy for “submodule” is used to access a Proxy for its deferred intermediate output via the `.output` attribute. Also on line five, the returned Proxy is intervened on, creating two new Proxys and Nodes through the overloaded slicing and setting methods. Finally on line six, the same intermediate value is placed in the “hidden.state” variable, and saved for use after model execution.

When exiting the tracing context, the NNSight object interleaves the traced operations to the model’s computation graph and executes the model. This process is performed by adding PyTorch hooks to all modules whose Envoy’s `.input` or `.output` attributes were accessed within the tracing context. These hooks are the interface between the intervention graph, formed from the deferred operations gathered while inside of the tracing context, and the model’s original computation graph. After interleaving, NNSight removes these hooks.

The Intervention Graph The Intervention Graph encapsulates the logic that should be interleaved with model’s original computation graph. The Graph is composed of individual Node objects, each representing a single operation to be executed during evaluation of the graph. Nodes have arbitrary positional and keyword arguments. These arguments may contain other Nodes which are considered *dependencies* of the source Node. Conversely, the source Node is a *listener* of its *dependency*. When a Node’s remaining *listeners* reaches zero, it is considered redundant and has its value cleared and its memory freed immediately.

Portions of the Intervention Graph are divided into sub-graphs depending on which module they rely on. These sub-graphs are then executed sequentially when their corresponding module’s hook is called during model execution. In other words, the root intervention Nodes created via Envoys (`.input` or `.output`) act as GOTO statements that transfer execution of the Intervention Graph, beginning from that Node up until they hit another root intervention Node. Modules may be executed more than once during interleaving, and therefore these sub-graphs may also be executed multiple times, just as you would a for loop.

For every Node, there is a corresponding Proxy that manages tracing operations on the underlying Node. Proxys achieve this by overriding builtin Python and PyTorch magic methods. These overrides

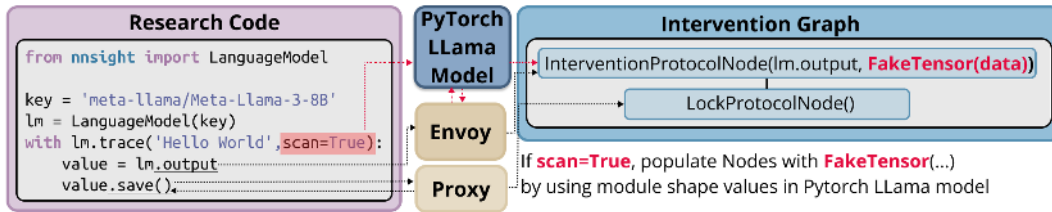


Figure 8: The components involved in creating nodes within the Intervention Graph. Module calls like `lm.output` are processed through Envoy, resulting in the creation of relevant ProtocolNodes like `InterventionProtocolNode()`. Everything else goes through Proxy layers to create other relevant nodes in the graph. All nodes are encapsulated within `InterventionProxy()`, which users see upon printing nodes. NNSight can scan the modules in the PyTorch model instance to gather related data shapes with a scan flag, creating FakeTensors that are included in the respective intervention graph nodes and returned to the user.

intercept calls to methods and functions that now create and add a new Node to the Intervention Graph, returning the Node’s corresponding Proxy. Chaining this process results in a completed Intervention Graph.

Protocols When composed into an Intervention Graph, Nodes are quite powerful on their own. However, a full-featured framework requires operations that can exert more control on the intervention process than the scope an individual Node can provide.

A Protocol is a special NNSight operation that enables these more powerful controls. A Protocol must define two methods: One that adds a Node of its type to the Intervention Graph, and another that executes that Node during interleaving. Rather than NNSight executing a Node’s target with its arguments, Protocols allow the Node to be passed the Protocol for execution. With full access to the Node, the Protocol can process its arguments, implement complex logic, and access the Intervention Graph itself. This access provides the Protocol a global view of the intervention process, which it can leverage by interacting with the attachments attribute on the Intervention Graph. Attachments are a dictionary on the Intervention Graph where Protocols can manage a global state throughout interleaving.

Two types of Protocols are highlighted below:

LockProtocol: When a Node has no more listeners, its value is cleared to free memory. Therefore this NO-OP operation never updates the remaining listeners of its dependencies, preventing a Node’s value from being deleted. Proxys implement a `.save()` method which creates a LockProtocol with the source Node as its only dependency, resulting in the Node’s value being available outside the tracing context.

GradProtocol: Many use cases require intervening on the backwards pass of a model rather than exclusively the forward pass. Accessing a Proxy’s `.grad` attribute creates a GradProtocol Node. When executed, GradProtocol adds a backwards hook to the underlying Tensor value. This hook injects the gradients into the GradProtocol Node during the backwards pass, allowing it to behave like any other Node.

Model Abstractions The NNSight class defines abstract convenience methods that subclasses can implement to extend common neural network architectures to the NNSight API. These methods include logic to load the model, which is called twice when initializing and executing an NNSight-based model from scratch. The first call happens upon initialization to create and populate the Envoy tree with a ‘meta’ version of the model. In a ‘meta’ model, the parameters of the network are not yet loaded: they exist as placeholders containing shape and datatype information. The second call occurs when loading and dispatching the true model parameters.

NNSight comes with a few of these subclasses pre-implemented. The most common subclass is the `LanguageModel`, for text-to-text sequence generation architectures. The `LanguageModel` integrates with Huggingface to load model configurations, tokenizers, and parameters from their online repository when provided with a repo id. Implementing the input processing methods uses

the model’s tokenizer to accept common forms of inputs and batch them together. When loading the ‘meta’ version of the model, the `LanguageModel` uses `AutoModelForCausalLM.from_config` from Huggingface transformers to download only the models configuration. Only upon dispatching does `LanguageModel` fetch the model parameters if they aren’t already cached.

Scanning and Validation NNsight offers two key debugging features integrated with the Tracer context.

The first is Scanning. When toggled, scanning performs an initial execution pass on the model in `FakeTensorMode`. During this pass, Hooks registered on each module update the input and output of their corresponding Envoys on the NNsight model with generated `FakeTensors`. This process makes valuable information available during tracing such as output shapes, dtypes, and device allocations.

When Scanning is enabled, Validation can be used to test interventions on `FakeTensors` as they are being added on the intervention graph. This acts as a sanity check to identify errors related to defined interventions before engaging in the model’s forward pass run.

Remote Execution and Session It is common for users to run multiple forward passes on a model as part of a larger experiment, such as in a LoRA training loop. NNsight offers an API that allows the creation of a Session context, where inner defined Tracer contexts are executed sequentially only after the Session context is exited. By encapsulating multiple Tracers in a Session, values obtained in earlier passes can be seamlessly referenced by later stages in the experiment without the need for manual saving.

When a Session is executed remotely, all forward passes defined within are sent as part of a single request to be executed. This can reduce wait times between Tracer calls by 1) minimizing the number of server requests and 2) eliminating the need to download and re-upload values referenced across Tracer contexts for subsequent calls that require them.

B.2 NDIF DESIGN

NDIF enables distributed execution of intervention graphs across multiple GPUs. The system architecture consists of three main components: a request handling layer, a model service layer, and a distributed execution engine. The current implementation builds on top of Ray (Moritz et al., 2018; Ray Team, 2022) as a distributed scheduler.

Request Processing and Graph Distribution When a user creates an intervention with NNsight, the system serializes the intervention graph into a custom JSON format and sends it to NDIF’s HTTP server front-end. The request handler places the graph in Ray’s object store and queues it for processing by the Request Service deployment. This service routes the request to the appropriate Model Service deployment based on the requested model.

For distributed models split across multiple GPUs, NDIF maintains separate Ray deployments for each model shard. The head shard (shard 0) receives the request first and distributes it to other shard deployments. All shards wait to receive the complete intervention graph before beginning execution.

Distributed Model Management NDIF loads large models using a sharding process. First, it creates a “meta model” containing only shape information without loading weights. It then attaches hooks to PyTorch’s `load_state_dict` functionality that convert modules into their parallel versions during loading. This allows NDIF to load model checkpoints shard-by-shard while maintaining a set memory limit for unsharded tensors at any given time.

Executing Distributed Interventions During execution, NDIF handles interactions between the distributed model and the intervention graph. When encountering distributed tensors (`DTensors`), NDIF records tensor placement information, converts `DTensors` to full tensors using `torch.distributed.gather` operations, injects the full tensors into the intervention graph, and then re-shards tensors after graph execution using saved placement information.

This approach allows the intervention graph to operate on complete tensors while maintaining memory efficiency. Only the head shard handles communication with the user, including websocket updates and returning saved values.

Co-tenancy and Resource Management NDIF supports sequential co-tenancy, allowing multiple users to access the same model instance without reloading parameters. Future implementations will enable parallel co-tenancy through batch grouping as follows: During tracing, intervention nodes record batch groups that specify tensor slices. During execution, the system extracts appropriate slices while preserving gradient propagation, enabling multiple users to share execution within a single forward pass.

C CODE COMPARISON

In Section 3.2, we discussed how NNSight can express any experiment that can be expressed with PyTorch, but that is also simplifies the design of custom experiments on large model internals compared to using standard PyTorch hooks. Here we provide an additional pair of code examples that implement a common interpretability technique called “activation patching” (Zhang & Nanda, 2024). Code Example 2 implements this experiment using PyTorch hooks, while Code Example 3 uses the NNSight API. Both code snippets implement the same intervention, causing the model to change its prediction for the base prompt from “Paris” to “Rome”.

```

1 from transformers import AutoTokenizer, AutoModelForCausalLM
2 model_id = "meta-llama/Meta-Llama-3.1-8B"
3 tokenizer = AutoTokenizer.from_pretrained(model_id, padding_side='left')
4 tokenizer.pad_token = tokenizer.eos_token
5 lm = AutoModelForCausalLM.from_pretrained(model_id)

7 edit_prompt = "The Colosseum is located in the city of"
8 base_prompt = "The Louvre is located in the city of"
9 batch = [edit_prompt, base_prompt]

11 edit_tok = 5
12 base_tok = 6

14 def hook_fn(module, input, output):
15     output[0][1, base_tok, :] = output[0][0, edit_tok, :]

17 layer = lm.model.layers[5]
18 hook = layer.register_forward_hook(hook_fn)
19 inputs = tokenizer([edit_prompt, base_prompt], return_tensors="pt", padding=True)
20 output = lm(**inputs)
21 hook.remove()

23 last = output["logits"][-1, -1].argmax()
24 prediction = tokenizer.decode(last)
25 print(prediction)

```

Code Example 2: An intervention implemented with standard PyTorch hooks. Lines 1-4 load a language model from huggingface, and lines 7-9 define two prompts. In lines 11-12, we note the token indices for the last token of the subjects in each prompt. Lines 14-15 define a PyTorch hook that replaces the hidden state in the base prompt with the hidden state from the edit prompt using the respective last-subject-token indices. The experiment is carried out in lines 17-21, and we collect and print the output of the model in lines 23-25.

```

1 from nnsight import LanguageModel
2 model_id = "meta-llama/Meta-Llama-3.1-8B"
3 lm = LanguageModel(model_id)

5 edit_prompt = "The Colosseum is located in the city of"
6 base_prompt = "The Louvre is located in the city of"
7 batch = [edit_prompt, base_prompt]

9 edit_tok = 5
10 base_tok = 6

12 layer = lm.model.layers[5]
13 with lm.trace(batch) as tracer:
14     layer.output[0][1, base_tok, :] = layer.output[0][0, edit_tok, :]
15     output = lm.output.save()

17 last = output["logits"][-1, -1].argmax()
18 prediction = lm.tokenizer.decode(last)
19 print(prediction)

```

Code Example 3: An intervention implemented with the NNSight API. This code defines the same intervention as in Code Example 2. Lines 1-3 load a language model from huggingface, and lines 5-7 define two prompts. In lines 9-10, we note the token indices for the last token of the subjects in each prompt. The experiment is carried out in lines 12-15, where we replace the hidden state in the base prompt with the hidden state from the edit prompt, using the respective last-subject-token indices. We collect and print the output of the model in lines 17-19.

Code Example 4 shows an NNSight implementation for attribution patching (Kramár et al., 2024), a method that involves both causal interventions and computation of gradients within the model.

```

1 def get_batch_states_and_grads(model, submodules, submod_names, batch, batch_target_ids):
2     hidden_states_cache = {}
3     grads_cache = {}
4
5     with model.trace(batch, remote=True) as tracer:
6         for submodule in submodules:
7             if 'neurons' in submod_names[submodule]:
8                 x = submodule.input
9             else:
10                x = submodule.output[0]
11                hidden_states_cache[submodule] = x.save()
12                grads_cache[submodule] = x.grad.save()
13                neg_log_likelihood = metric_nll(model, batch_target_ids).save()
14                neg_log_likelihood.sum().backward()
15
16     hidden_states = {k : v.value[:, token_idx] for k, v in hidden_states_cache.items()}
17     grads = {k : v.value[:, token_idx] for k, v in grads_cache.items()}
18     return hidden_states, grads, neg_log_likelihood

```

Code Example 4: An NNSight implementation for attribution patching (Kramár et al., 2024)

In Code Example 5, we show an example of how to implement and train a LoRA (Hu et al., 2021) adapter using a session context with remote execution via NDIF. Code Examples 6 and 7 demonstrate interventions on the gradients of model computations. And in Code Example 8, we show an what training a linear probe remotely via NDIF might look like to predict output of layer 1 using the output of layer 0.

```

1 import torch
2
3 from nnsight.envoy import Envoy
4 from torch.utils.data import DataLoader
5
6 # We will define a LORA class.
7 # The LORA class call method operations are simply traced like you would normally do in a .
8 # trace.
9 class LORA(nn.Module):
10     def __init__(self, module: Envoy, dim: int, r: int) -> None:
11         """Init.
12
13         Args:
14             module (Envoy): Which model Module we are adding the LORA to.
15             dim (int): Dimension of the layer we are adding to (This could potentially be
16             auto populated if the user scanned first so we know the shape)
17             r (int): Inner dimension of the LORA
18         """
19         super(LORA, self).__init__()
20         self.r = r
21         self.module = module
22         self.WA = torch.nn.Parameter(torch.randn(dim, self.r), requires_grad=True).save()
23         self.WB = torch.nn.Parameter(torch.zeros(self.r, dim), requires_grad=True).save()
24
25     # The Call method defines how to actually apply the LORA.
26     def __call__(self, alpha: float = 1.0):
27         """Call.
28
29         Args:
30             alpha (float, optional): How much to apply the LORA. Can be altered after
31             training for inference. Defaults to 1.0.
32         """
33
34         # We apply WA to the first positional arg (the hidden states)
35         A_x = torch.matmul(self.module.input[0][0], self.WA)
36         BA_x = torch.matmul(A_x, self.WB)
37
38         # LORA is additive
39         h = BA_x + self.module.output
40
41         # Replace the output with our new one * alpha
42         # Could also have been self.module.output[:] = h * alpha, for in-place
43         self.module.output = h * alpha
44
45     def parameters(self):
46         # Some way to get all the parameters.
47         return [self.WA, self.WB]
48
49 # get the model
50 llama = LanguageModel("meta-llama/Meta-Llama-3.1-70B")
51
52 # We need the token id of the correct answer.

```

```

50 answer = " Paris"
51 answer_token = llama.tokenizer.encode(answer)[1]
52 # Inner LORA dimension
53 lora_dim = 4

55 # Module to train LORA on
56 module = llama.model.layers[-1].mlp

60 # The LORA object itself isn't transmitted to the server. Only the forward / call method.
61 # The parameters are created remotely and never sent only retrieved
62 with llama.session(remote=True) as session:

64     # Create dataset of 100 pairs of a blank prompt and the " Paris " id
65     dataset = [["_ ", answer_token]] * 100

67     # Create a dataloader from it.
68     dataloader = DataLoader(dataset, batch_size=10)

70     # Create our LORA on the last mlp
71     lora = LORA(module, dim, lora_dim)

73     # Create an optimizer. Use the parameters from LORA
74     optimizer = torch.optim.AdamW(lora.parameters(), lr=3)

76     # Iterate over dataloader using .iter.
77     with session.iter(dataloader, return_context=True) as (batch, iterator):

79         prompt = batch[0]
80         correct_token = batch[1]

82         # Run .trace with prompt
83         with llama.trace(prompt) as tracer:

85             # Apply LORA to intervention graph just by calling it with .trace
86             lora()

88             # Get logits
89             logits = llama.lm_head.output

91             # Do cross entropy on last predicted token and correct_token
92             loss = torch.nn.functional.cross_entropy(logits[:, -1], batch[1])
93             # Call backward
94             loss.backward()

96             # Call methods on optimizer. Graphs that arent from .trace (so in this case session
97             # and iterator both have their own graph) are executed sequentially.
98             # The Graph of Iterator here will be:
99             # 1.) Index batch at 0 for prompt
100            # 2.) Index batch at 1 for correct_token
101            # 3.) Execute the .trace using the prompt
102            # 4.) Call .step() on optimizer
103            optimizer.step()
104            # 5.) Call .zero_grad() in optimizer
105            optimizer.zero_grad()
106            # 6.) Print out the lora WA weights to show they are indeed changing
            iterator.log(lora.WA)

```

Code Example 5: Using NNSight to train a LORA with remote execution, the Session context, and iterative interventions. NNSight supports creating parameters and optimizers remotely. Using ‘model.trace’ as a forward pass, a dataset can be iterated over to optimize the remote parameters. The normal workflow of calculating a loss, executing a backwards pass, and taking optimizations steps is expressed just as if this were code executing on the user’s local workstation. After executing the training loop, the optimized parameters are sent back to the user’s workstation to be used locally, or to be sent back to the remote server for future applications of parameters.

```

1 import nnsight
2 from nnsight import NNSight
3 from collections import OrderedDict
4 import torch

6 input_size = 5
7 hidden_dims = 10
8 output_size = 2

10 net = torch.nn.Sequential(
11     OrderedDict(

```

```

12         [
13             ("layer1", torch.nn.Linear(input_size, hidden_dims)),
14             ("layer2", torch.nn.Linear(hidden_dims, output_size)),
15         ]
16     )
17 ).requires_grad_(False)
18 tiny_model = NNSight(net)
19 input = torch.rand((1, input_size))
21 with tiny_model.trace(input):
23     # We need to explicitly have the tensor require grad
24     # as the model we defined earlier turned off requiring grad.
25     tiny_model.layer1.output.requires_grad = True
27     # We call .grad on a tensor Proxy to communicate we want to store its gradient.
28     # We need to call .save() since .grad is its own Proxy.
29     layer1_output_grad = tiny_model.layer1.output.grad.save()
30     layer2_output_grad = tiny_model.layer2.output.grad.save()
32     # Need a loss to propagate through the later modules in order to have a grad.
33     loss = tiny_model.output.sum()
34     loss.backward()
36 print("Layer 1 output gradient:", layer1_output_grad)
37 print("Layer 2 output gradient:", layer2_output_grad)

```

Code Example 6: Applying backpropagation and accessing gradients with respect to a loss.

```

1 import nnsight
2 from nnsight import NNSight
3 from collections import OrderedDict
4 import torch
6 input_size = 5
7 hidden_dims = 10
8 output_size = 2
10 net = torch.nn.Sequential(
11     OrderedDict(
12         [
13             ("layer1", torch.nn.Linear(input_size, hidden_dims)),
14             ("layer2", torch.nn.Linear(hidden_dims, output_size)),
15         ]
16     )
17 ).requires_grad_(False)
18 tiny_model = NNSight(net)
19 input = torch.rand((1, input_size))
21 with tiny_model.trace(input):
23     # We need to explicitly have the tensor require grad
24     # as the model we defined earlier turned off requiring grad.
25     tiny_model.layer1.output.requires_grad = True
27     tiny_model.layer1.output.grad[:] = 0
28     tiny_model.layer2.output.grad = tiny_model.layer2.output.grad * 2
30     layer1_output_grad = tiny_model.layer1.output.grad.save()
31     layer2_output_grad = tiny_model.layer2.output.grad.save()
33     # Need a loss to propagate through the later modules in order to have a grad.
34     loss = tiny_model.output.sum()
35     loss.backward()
37 print("Layer 1 output gradient:", layer1_output_grad)
38 print("Layer 2 output gradient:", layer2_output_grad)

```

Code Example 7: Zeroing out the grad of one layer, and doubling the grad of a second layer.

```

1 import torch
2 from torch.utils.data import DataLoader
3 from nnsight import LanguageModel, list, log
5 model = LanguageModel("meta-llama/Meta-llama-3.1-405B")
7 with model.session(remote=True) as session:
8     features = 4096
9     probe = torch.nn.Linear(features, features).save()

```

```
10 dataset = DataLoader(  
11     ["some", "text", "to", "train", "on"], batch_size=2, shuffle=True  
12 )  
13 optimizer = torch.optim.Adam(probe.parameters(), lr=0.003)  
  
15 for epoch in list(range(50)):  
16     for batch in dataset:  
17         with model.trace(batch) as tracer:  
18             input = model.model.layers[0].output[0]  
19             probe = probe.to(input.device)  
20             prediction = probe(input)  
21             output = model.model.layers[1].output[0]  
22             loss = torch.nn.functional.mse_loss(prediction, output)  
23             loss.backward()  
  
25         optimizer.step()  
26         optimizer.zero_grad()  
  
28     log(loss)  
29     log(probe.weight)
```

Code Example 8: Sample code showing the remote training of a linear probe using NDIF.

D PERFORMANCE

In this section, we report additional results evaluating the performance of NNsight and NDIF in various settings. These include setup and runtime across different model sizes of OPT (Zhang et al., 2022), comparing the setup and runtime of HPC and NDIF for Llama-3.1 models (Dubey et al., 2024), and the response time for NDIF requests as a function of the number of users.

D.1 HPC vs. NDIF

Table 2: Setup-time and runtime comparison of HPC and NDIF across OPT parameter sizes

OPT	Parameters	Setup Time	Runtime
HPC	125m	0.635 ± 0.035	0.021 ± 0.001
	350m	0.847 ± 0.058	0.041 ± 0.001
	1.3b	1.242 ± 0.016	0.072 ± 0.004
	2.7b	1.866 ± 0.003	0.123 ± 0.011
	6.7b	3.474 ± 0.023	0.257 ± 0.008
	13b	5.971 ± 0.021	0.429 ± 0.018
	30b	11.338 ± 0.073	0.913 ± 0.016
	66b	23.697 ± 0.079	1.889 ± 0.040
NDIF	125m	0.149 ± 0.008	0.541 ± 0.123
	350m	0.304 ± 0.015	0.588 ± 0.178
	1.3b	0.409 ± 0.213	0.627 ± 0.099
	2.7b	0.358 ± 0.054	0.494 ± 0.085
	6.7b	0.317 ± 0.011	0.550 ± 0.080
	13b	0.370 ± 0.026	0.773 ± 0.136
	30b	0.343 ± 0.013	1.295 ± 0.104
	66b	0.423 ± 0.095	2.739 ± 0.154

Table 3: Runtime comparison of NNsight using execution on an HPC and remote execution via NDIF.

Framework	Activation Patching	
	Llama-3.1-8B	Llama-3.1-70B
NNsight (HPC)	0.333 ± 0.013	1.992 ± 0.056
NNsight (NDIF)	0.998 ± 0.134	2.369 ± 0.117

Table 4: Time in seconds to load the model into memory.

Framework	Loading Weights	
	Llama-3.1-8B	Llama-3.1-70B
NNsight (HPC)	5.991 ± 0.192	43.617 ± 1.118
NNsight (NDIF)	0.518 ± 0.005	0.695 ± 0.056

D.2 LOAD-TESTING NDIF FOR CONCURRENT USERS

In this section, we assess the performance and scalability of NDIF with an increasing number of concurrent users (and thereby requests). We use Llama-3.1-8B (Dubey et al., 2024) as the model being served on NDIF for this evaluation, which was hosted on a 48 GB RTX 6000 Ada GPU.

To simulate a single user, we construct a NNsight request with a prompt containing up to 24 tokens that accesses and saves the output of a layer of Llama-3.1-8B selected uniformly at random. Code

Example 9 shows how we generate sample prompts and requests for a single user, saving the response time of the request sent to and returned by NDIF.

```

1 def layer_selector(self) -> None:
2     """Simulate selecting activations of an intermediate layer."""
3     start_time = time.time()
4     try:
5         layer : int = random.randint(0, self.n_layers - 1)
6         query : str = 'hello ' * random.randint(1, 24)
7         with self.model.trace(query, remote=True):
8             output = self.model.model.layers[layer].output.save()
9     finally:
10        elapsed_time = time.time() - start_time
11        # Append to a list visible locally to this process
12        self.user.environment.request_times.append(elapsed_time)

```

Code Example 9: Code to generate NDIF requests and measure the response time for many users.

Then, for $N \in \{1, \dots, 100\}$, we simulate N concurrent users submitting requests to the NDIF server, all running as separate processes, and record the response time for each user’s request.

The results can be seen in Figure 9, where we plot the response time of the NDIF system as a function of the number of concurrent users. We find that as the number of users increases, the increase in the median response time (shown in black) is approximately linear. Furthermore, as more concurrent users access the system, variance in response time increases.

This experiment was done using an implementation of the system that creates a queue for each subsequent user, and runs multiple forward passes (one for each user). There are future plans to allow true parallel execution of user requests in the same batch.

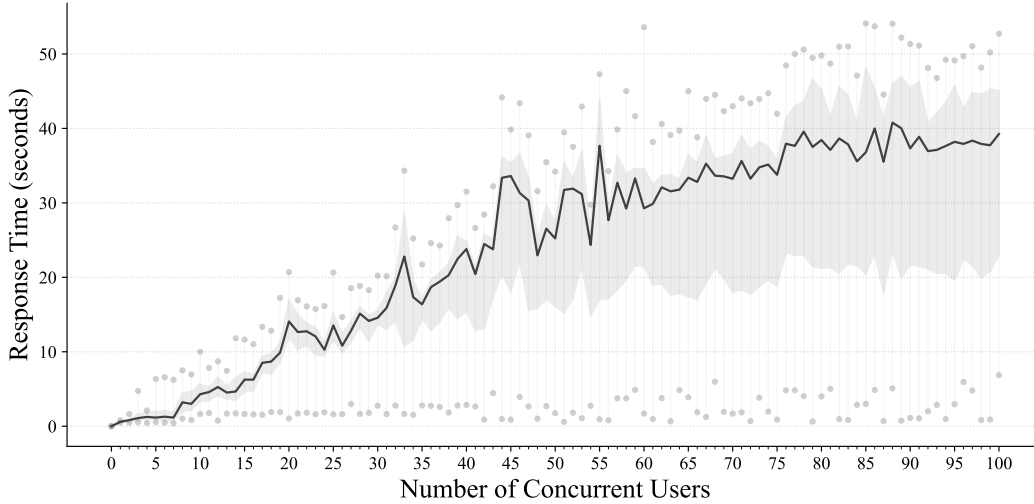


Figure 9: Speed and robustness of NDIF response time to changes in concurrent user request count. Tests simulated $N = 1$ to 100 concurrent users on Llama-3.1-8B requesting outputs from uniformly random intermediate layers over a 30-second window. The gray area shows 25% and 75% quantiles. Points show the minimum and maximum times for each user count. Line plot shows median response time.

E EQUIVALENCE OF THEANO’S MANY-TO-MANY AND NNSIGHT’S MANY-TO-ONE APPLY NODES

We demonstrate that Al-Rfou et al. (2016)’s many-to-many computation graph structure—where an apply node can output multiple variable nodes—is computationally equivalent to a graph in which each apply node outputs only a single variable node. This equivalence holds by considering the many-to-many structure as a special case of a many-to-one structure, where the single output is a tuple (or concatenation) of variable nodes.

Let $a^{(\text{in})}$ denote an apply node in the many-to-many setting, and let $v_1, \dots, v_m$ represent the m variable node outputs of $a^{(\text{in})}$. Suppose these variable nodes serve as inputs to n downstream apply nodes $a_1^{(\text{out})}, \dots, a_n^{(\text{out})}$ in the computation graph.

We now construct a new graph that adheres to the many-to-one structure, where each apply node has exactly one output. Let $b^{(\text{in})}$, v , and $b_1^{(\text{out})}, \dots, b_n^{(\text{out})}$ denote corresponding nodes in the transformed graph, defined as follows:

1. The apply node $b^{(\text{in})}$ outputs a single variable node v , which is defined as the concatenation (or tuple) of the original outputs: $v = (v_1, \dots, v_m)$.
2. Each downstream apply node $b_i^{(\text{out})}$ takes as input v and any other variables that were inputs to $a_i^{(\text{out})}$ in the original graph. The node $b_i^{(\text{out})}$ performs the same computation as $a_i^{(\text{out})}$, but accesses the necessary components of v corresponding to its original inputs. Specifically, if $a_i^{(\text{out})}$ originally took inputs $v_{i_1}, \dots, v_{i_k}$, then $a_i^{(\text{out})}$ extracts $v_{i_1}, \dots, v_{i_k}$ from v and processes them accordingly.

This process generalizes inductively to any depth of the computation graph. In the event that one of the output variable nodes v_i needs to be used elsewhere in the computation graph (e.g., as input to a node that does not directly accept v), the transformation can be handled by introducing an additional apply node, $b^{(\text{extract})}$. This node takes v as input and outputs v_i , ensuring that the structure of the original computation graph is maintained.

Figure 10 shows an example of how getters and setters interact with an intervention graphs variable and apply nodes on an abstract computation graph.

Figure 10: Augmenting the computation graph C (pink graph on the left) with an intervention graph (green, right). (a) A getter edge brings the output of a variable node v_1 from the computation graph to an apply node a'_1 in the intervention graph. (b) A setter edge sends the output of v'_2 in the intervention graph back to a_3 in the computation graph.